

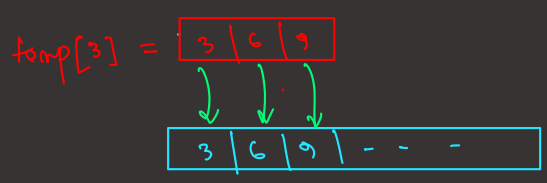
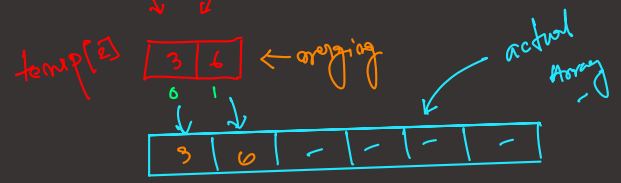
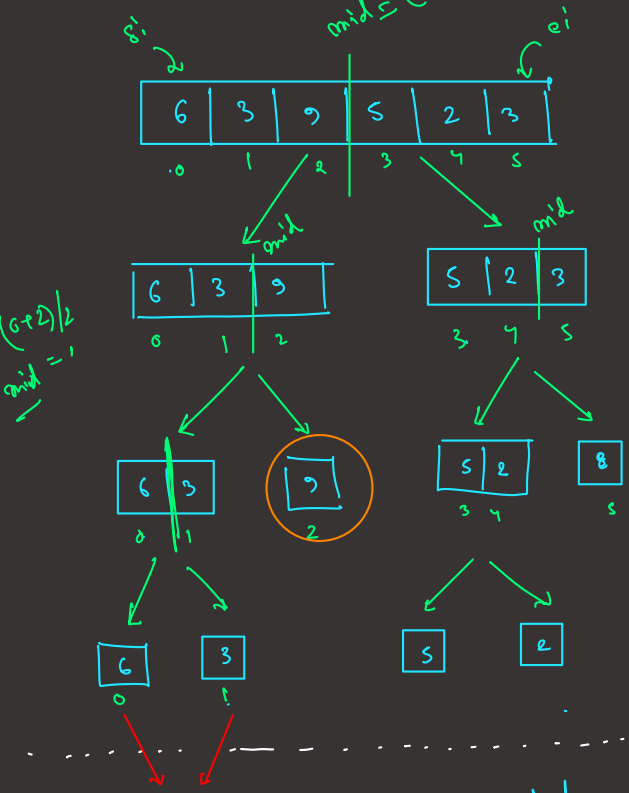
day 17 2) Divide and Conquer

- ① Merge Sort
- ② Quick Sort

$0 \rightarrow (5+6)/2$

$(0+5)/2 = 2$
 $mid = (si + ei)/2$

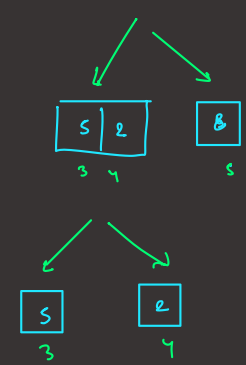
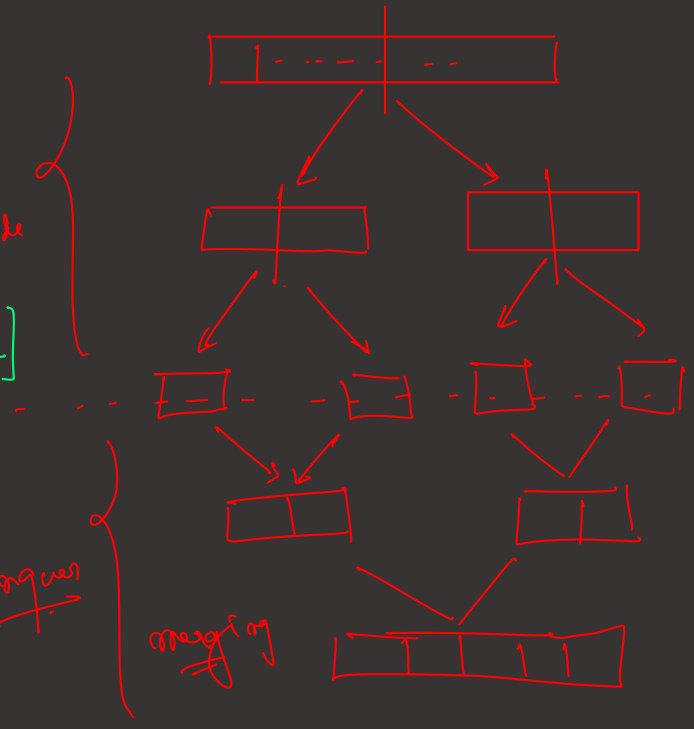
$[mid = si + (ei - si)/2]$



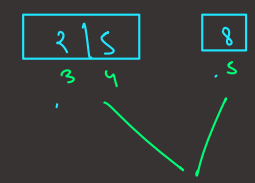
divide

conquer

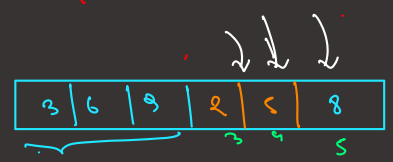
merging



temp = [2, 5]



temp[3] = [2, 5, 8]

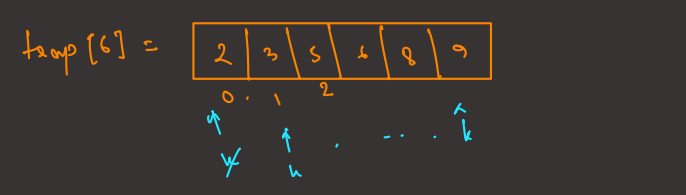


$i = s_i$ to mid

$j = mid+1$ to e_i

Space Complexity $O(n)$
 auxiliary array

Time Complexity = $O(n \log n)$



$temp[e_i - s_i + 1]$

Code

```
void mergeSort(arr, si, ei) {
    if (si > ei) return;
    mid = (si + ei) / 2;
    mergeSort(arr, si, mid);
    mergeSort(arr, mid + 1, ei);
    merge(arr, si, mid, ei);
}
```

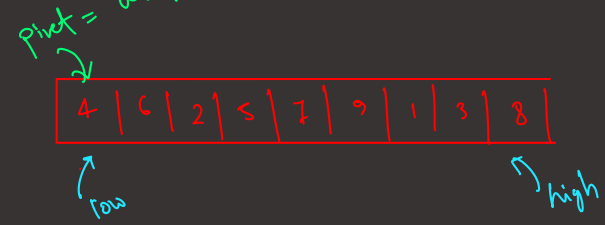
```
merge(arr, si, mid, ei) {
    temp[e_i - s_i + 1];
    i = s_i; j = mid + 1; k = 0;
    while (i <= mid && j <= ei) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++; k++;
        }
        else {
            temp[k] = arr[j];
            j++; k++;
        }
    }
    while (i <= mid) {
        temp[k] = arr[i];
        i++; k++;
    }
    while (j <= ei) {
        temp[k] = arr[j];
        j++; k++;
    }
}
```

```
// copy
for (k = 0; i = s_i; k < temp.length; i++, k++) {
    arr[i] = temp[k];
}
```

}

initial arr[0]

Quick Sort



$T.C = O(n \log n) \leftarrow \text{Avg case}$
 $O(n^2) \leftarrow \text{worst case}$

- pivot element
- first element ✓
 - last element
 - mid element
 - any random element
 - median

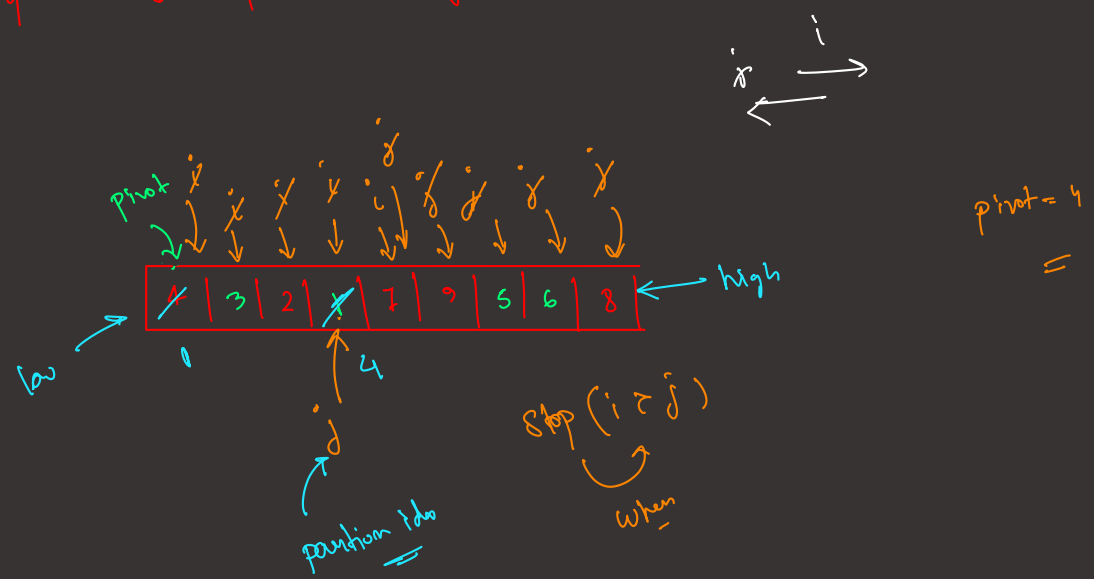
$\text{low to (pivot-1)} \leq \text{pivot} < \text{pivot+1 to high}$

$S.C = O(1)$

① find the correct position for the pivot.
 $\text{pivot} = \text{partition index}()$

Code

```
quickSort(arr, low, high) {  
    if (low >= high) return;  
    int partition = partition(arr, low, high);  
    quickSort(arr, low, partition-1);  
    quickSort(arr, partition+1, high);  
}
```



$\text{arr}[i] \leq \text{pivot} < \text{arr}[j]$

```

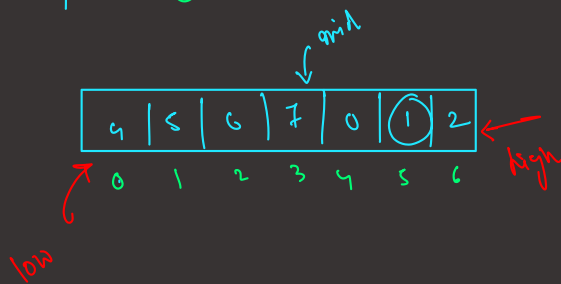
Code
int partition(arr, low, high) {
    pivot = arr[low];
    i = low;
    j = high;
    while (i < j) {
        while (arr[i] ≤ pivot && i ≤ high) {
            i++;
        }
        while (pivot < arr[j] && j > low) {
            j--;
        }
        if (i < j) {
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[j], arr[low]);
    return j;
}

```

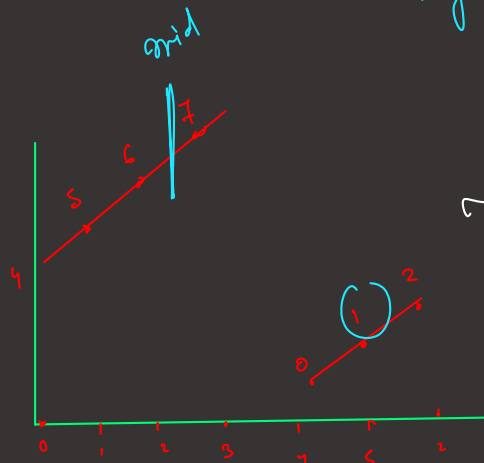
Q Search in a Rotated Sorted Array

Search for 1

4 5 6 7 0 1 2



6 7 0 1 2 3 4 5



$$T.C = O(\log n)$$

BS
 key == mid $\rightarrow r = mid$
 key < mid
 l.h.
 key > mid
 R.h.

